MARS ROVER MANIPAL
RESEARCH AI TASKPHASE
Report 7

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Review questions

1.1 What is the Kernel Trick?

The Kernel Trick allows us to apply a linear algorithm (like SVM) to nonlinear data without ever explicitly transforming the data to a higher-dimensional space.

- Let us say we have a mapping function $\phi(x)$.
- This function transforms the input data x into a higher-dimensional space.
- An SVM (and many other algorithms) only needs dot products of feature vectors.

$$\phi(x_i) \cdot \phi(x_j)$$

- But computing $\phi(x)$ explicitly can be very expensive or impossible (since it might be infinite-dimensional).
- Instead, we define a kernel function $K(x_i, x_j)$ such that

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

- This is the **Kernel Trick**.
- It allows us to compute the dot product in a higher-dimensional space without ever computing the mapping itself.

1.2 What is the loss for SVM and the trade-off related to it?

The loss used in SVM is the **Hinge Loss**.

$$L(y, f(x)) = max(0, 1 - y \cdot f(x))$$

where $f(x) = wx + b$.

Interpretation:

- If a sample is correctly classified and far enough from the margin ($y \cdot f(x) \geq 1$), loss = 0.
- If it is within the margin or misclassified ($y \cdot f(x) < 1$), loss increases linearly.

The overall SVM optimization problem can thus be written as

$$min_{w,b} \frac{1}{2}||w||^2 + C \sum_{i=1}^{n} max(0, 1 - y_i(wx_i + b))$$

The trade-off is controlled by the parameter **C**.

- If C is large, it penalizes misclassification heavily. The model focuses on minimizing training errors, which may lead to a narrow margin and overfitting.
- If C is small, it allows some errors and focuses on maximizing margin. The model gets wider margin, may underfit if too small.

1.3 What are the different types of margins for SVM?

In Support Vector Machines (SVM), the margin is the distance between the decision boundary (hyperplane) and the closest data points from each class. These closest points are called support vectors. SVM aims to maximize this margin, leading to better generalization on unseen data.

Types of margins

1. **Hard margin**
    - No misclassification allowed. All points must be correctly separated and outside the margin.
    - Used when the data is perfectly linearly separable.
2. **Soft margin**
    - Some points are allowed to be within or even on the wrong side of the margin (controlled by slack variables).
    - Used when data is not perfectly separable (real-world data).
3. **Functional margin**
    - Measure of confidence in the classification.
    - Can be scaled arbitrarily.
4. **Geometric margin**
    - Actual perpendicular distance from point to hyperplane.
    - Scale-invariant.

1.4 What are the types of clustering algorithms?

Clustering is an unsupervised machine learning technique used to group similar data points together without any predefined labels.
Each group is called a cluster, and the goal is to ensure that:

- Points within a cluster are as similar as possible, and
- Points across clusters are as different as possible.

Types of clustering algorithms

1. **Partition-based**
    - Split data into k distinct non-overlapping clusters based on distance.
    - ***Examples:***
        - K-Means
        - K-Medoids (PAM)
2. **Heirarchal**
    - Build a hierarchy (tree/dendrogram) of clusters.
    - ***Examples:***
        - Agglomerative (bottom-up)
        - Divisive (top-down)
3. **Density-based**
    - Form clusters where data points are closely packed together; can detect arbitrary shapes and outliers.
    - ***Examples:***
        - DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
        - OPTICS
4. **Model-based**
    - Assume data is generated from a mix of probabilistic models and infer parameters.
    - ***Examples:***
        - Gaussian Mixture Models (GMM)
5. **Grid-based**
    - Divide space into grids and cluster based on data density within grids.
    - ***Examples:***
        - STING (Statistical Information Grid)
        - CLIQUE

1.5 What is RBF?

RBF stands for Radial Basis Function. It is a type of kernel function that measures how similar two points are based on their distance in feature space.
For two points x and x':

$$K(x, x') = e^{-\frac{||x-x'||^2}{2\sigma^2}}$$

Here,

- $||x - x'||$ is the Euclidean distance between the two points.

- σ or $\gamma = \frac{1}{2\sigma^2}$ is the width parameter, which controls how quickly similarity drops off with distance.

RBF converts distance into similarity.

- For points close together, $||x - x'||$ is small, $K(x, x') \approx 1$ (very similar)
- For points far apart, $||x - x'||$ is large, $K(x, x') \approx 0$ (not similar)

1.6 What are the metrics for K-Means?

1. **Internal evaluation metrics**
    - ***Inertia***
        - Measures how far each point is from its cluster centroid.

$$Inertia = \sum_{i=1}^{k} \sum_{x \in C_i} \left|\left|x - \mu_i\right|\right|^2$$

        - ***Where:***
            - C_i: cluster i
            - μ_i: centroid of cluster i
        - A lower inertia is better as it results in tighter clusters.
        - But it decreases with increasing k, so it can't be used alone to pick k.
    - ***Silhouette score***
        - Measures how similar a point is to its own cluster vs other clusters.

$$s(i) = \frac{b(i) - a(i)}{max(a(i), b(i))}$$

        - ***Where:***
            - $a(i)$: average distance of point i to others in the same cluster
            - $b(i)$: average distance of point i to the nearest other cluster
        - ***Meaning of silhouette scores***
            - ≈ 1.0: well-clustered (clearly separated)
            - ≈ 0: overlapping clusters
            - < 0: misclassified point (closer to another cluster)
        - Usually, a higher silhouette score is better.
    - ***Calinski–Harabasz Index (Variance Ratio Criterion)***
        - Measures ratio of between-cluster dispersion to within-cluster dispersion:
            - $CH = \frac{between-cluster\ variance}{within-cluster\ variance} \times \frac{n-k}{k-1}$
        - A higher CH usually means better defined clusters.

- ***Davies–Bouldin Index***
  - Measures average similarity between each cluster and its most similar one.
  - $DB = \frac{1}{k}\sum_{i=1}^{k} max_{j\neq 1}\frac{s_i+s_j}{d_{ij}}$
  - ***Where:***
    - s_i: average distance of points in cluster i to its centroid
    - d_{ij}: distance between centroids of clusters i and j
  - A lower DB usually means better clusters due to less overlap.
- ***Dunn Index***

$$D = \frac{min\ inter-cluster\ distance}{max\ intra-cluster\ distance}$$

  - A higher Dunn index means better separation between clusters.

2. External evaluation metrics

- ***Adjusted Rand Index (ARI)***
  - Measures pairwise agreement between true labels and cluster labels.
  - Range is -1 to 1.
  - Higher is better.
- ***Normalized Mutual Information (NMI)***
  - Measures shared information between labels and clusters.
  - Range is 0 to 1.
  - Higher is better.
- ***Homogeneity Score***
  - Each cluster contains only members of a single class.
  - Range is 0 to 1.
  - Higher is better.
- ***Completeness score***
  - All members of a class are assigned to the same cluster.
  - Range is 0 to 1.
  - Higher is better.
- ***V-Measure***
  - Harmonic mean of homogeneity & completeness.
  - Range is 0 to 1.
  - Higher is better.

1.7 What are other dimensionality reduction techniques?

1. **Linear methods**
    - Preserve linear relationships.
    - ***Examples:***
        - PCA (Principal Component Analysis)
        - LDA (Linear Discriminant Analysis)
        - ICA (Independent Component Analysis)
        - FA (Factor Analysis)
2. **Nonlinear methods (Manifold Learning)**
    - Preserve nonlinear geometry.
    - ***Examples:***
        - t-SNE (t-distributed Stochastic Neighbor Embedding)
        - UMAP (Uniform Manifold Approximation and Projection)
        - Isomap (Isometric Mapping)
        - LLE (Locally Linear Embedding)
3. **Randomized / Feature-based methods**
    - Use random projections or feature selection.
    - ***Examples:***
        - Random Projection
        - Autoencoders (Neural Networks)
        - Feature Agglomeration

1.8 What is the difference between standardization and normalization?

1. **Standardization (Z-score scaling)**

$$z = \frac{x - \mu}{\sigma}$$

    - ***Where:***
        - x: original value of the feature
        - μ: mean of the feature
        - σ: standard deviation of the feature
    - Transforms data so that it has mean = 0 and standard deviation = 1.
    - Preferred for distance-based and gradient-based algorithms:
        - Linear / Logistic Regression
        - SVM
        - K-Means
        - PCA

2. **Normalization (min-max scaling)**

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

   - x_{max} and x_{min} are the maximum and minimum values of the feature.
   - Transforms data to a fixed range, usually [0, 1].
   - Works best when we need bounded values:
     - Neural networks (when activation functions like sigmoid or tanh are used)
     - Image pixel values (0–255 to 0–1)
     - Algorithms that compute distances (like KNN)

2. PCA (Principal Component Analysis)

2.1 Introduction and definition

- PCA is a dimensionality reduction technique.
- It tries to represent high-dimensional data in fewer dimensions while preserving as much of the variance (information) as possible.

2.1.1 What PCA does

- Finds directions (axes) in the data where variance is maximum.
- Rotates the coordinate system to align with these directions.
- Projects the data onto the top k of these directions (components).

So instead of using all original features, we only use these top few components.

2.2 Math involved in PCA

Let us assume our data matrix is X with n samples (rows) and p features (columns).

1. **Standardize the data**
   - Make every feature have mean = 0 and variance = 1.
   - For this, we can use Z-score scaling.

$$X_{scaled} = \frac{X - \mu_X}{\sigma_X}$$

2. **Compute covariance matrix**
   - This matrix shows how features vary with each other.

$$\Sigma = \frac{1}{n-1} X^T_{scaled} X_{scaled}$$

3. **Find eigenvectors and eigenvalues of the covariance matrix**

$$\Sigma v_i = \lambda_i v_i$$

   - ***Where:***
     - v_i: eigenvector (principal component direction)
     - λ_i: eigenvalue (amount of variance explained)
4. **Sort eigenvectors by descending eigenvalues**
   - The larger the eigenvalue, the more important that direction.
5. **Choose top *k* components**
   - Keep the first k eigenvectors as they define the new feature space.
6. **Project data**

$$X_{reduced} = X_{scaled} \times V_k$$

   - V_k are the top k eigenvectors.

2.3 Where PCA is used

PCA is used in these cases:

- When there are many correlated features.
- For visualization of high-dimensional data (2D/3D).
- To speed up ML algorithms.

PCA is avoided when:

- Feature meaning is crucial (PCA mixes them into abstract components).
- Data is not linearly correlated (try t-SNE or UMAP instead).